How to Approach Behavior-Driven Development (BDD) with Conformiq Visualizer

Behavior-Driven Development (BDD) is an Agile software development approach that emerged from the Test-Driven Development (TDD) methodology. It encourages collaboration among different stakeholders, to align their understanding of a software application. BDD introduces the core practice of the "3 Amigos concept", which involves the business, the development, and the testing/QA teams. This trio discusses the user story's acceptance criteria to achieve a shared perception of the task, defining what to build, how to test it, and the expected behavior.

The 3 amigos strategy offers benefits more than just improved communication; it really helps clear up requirements and sort out potential issues early on. By bringing different perspectives together at the same table, each participant can share their viewpoint, adding up to a much clearer big picture. This vision is what inspired the creation of **Conformiq Visualizer**, a powerful tool designed to foster collaboration. Its features allow for building and visually representing requirements in BPMN, and automating scenarios in a shared space.

Conformiq Visualizer makes it easier for the three amigos to understand and verify the application's behavior against business requirements. It ensures that the scenarios defined during BDD sessions are directly translated into executable test scripts, promoting accuracy and efficiency. In this article, we will review (at a high level) how you can tackle a BDD project with the use of Conformiq Visualizer.

Table of Contents

1. Three Main Issues that BDD can solve
2. Using Conformiq Visualizer for your BDD projects
   - 2.1. Define your System's Behavior
   - 2.2. Create Models in Conformiq Visualizer
   - 2.3. Export your Test Case Scenarios
   - 2.4. Insert Datasets into your Models
   - 2.5. Create Test Automation Files
   - 2.6. Collaborate with your Team
3. Overcoming BDD Deployment Challenges

Three Main Issues that BDD can solve

Behavior-driven development can address several common issues in software development. Overall, it serves as a bridge between technical and non-technical stakeholders, similar to a universal translator in a sci-fi film, which ensures that everyone understands each other perfectly. Here are 3 significant problems that BDD can help you solve:

1. Miscommunication Between Team Members

Problem	BDD Solution
Miscommunication between technical members (such as product managers, developers, UX designers, and quality assurance professionals) can lead to software that does not meet the requirements or expectations of the business and its users. Having a misunderstanding about a solution's functionality can result in costly reworks and delays.	BDD emphasizes collaboration and communication through the use of a common, domain-specific language that is understandable by all stakeholders. The Scenarios are written in plain language (using the ***Given***-***When***-***Then*** format), which helps everyone on the team understand exactly what the software should do. This approach reduces ambiguity and ensures that all team members have a clear, consistent understanding of the project requirements and expected outcomes.

2. Difficulty in Managing Changing Requirements

Problem	BDD Solution
In many development processes, changing requirements can disrupt the workflow, leading to scope creep and projects that are delivered late or over budget. Any company that wants to adapt to these changes can find it very challenging, especially if the initial requirements and their impact are not clearly understood or communicated.	By defining behaviors as part of the requirements process, BDD allows teams to focus on *what the software should accomplish* rather than *how it should be implemented*. This makes it easier to adapt when requirements change, as the impact of changes on existing behaviors can be assessed more clearly. Scenarios can be updated collaboratively to reflect new requirements, providing a dynamic and responsive approach to development.

3. Quality Assurance and Regression Issues

Problem	BDD Solution
Ensuring high-quality software delivery, especially with complex applications, can be overwhelming. Traditional testing methods may leave gaps in test coverage or may not adequately ensure that the software meets business needs. Furthermore, regression issues can easily arise when new features disrupt existing functionalities.	BDD integrates automated testing as a core part of the development process. Since scenarios written in BDD directly translate into automated tests, this approach ensures that all critical behaviors are tested from the start. These automated tests are run frequently, allowing teams to catch and fix errors early. As new features are added or existing features are modified, the BDD tests can help ensure that changes do not break existing functionality, thus reducing the risk of regressions.

Overall, BDD aligns software development with business needs by promoting better communication, adapting to changes smoothly, and ensuring a high level of quality and test coverage throughout the development lifecycle.

Conformiq Visualizer is a tool designed to transform the way software testing is done by automatically designing and generating high-quality test cases from models created within the software. This model-based approach aids in implementing BDD by allowing teams to create visual models that represent the expected behavior of the system.

Using Conformiq Visualizer for your BDD projects

Alright, but how can you address it? In this section, we will present a general overview of how you might deploy a Behavior Driven Development approach for your project, by using Conformiq Visualizer. Remember, with Conformiq Visualizer, your testing process will get much simpler and more efficient. So, let's break up the main steps that you should take to address your next BDD project:

> *For detailed information, please visit our website here*

1. Define your System's Behavior

- **Gather Requirements:** Start by gathering the behavior requirements from stakeholders. In BDD, these are often expressed in simple language that describes the expected behavior of the system.
- **Write User Stories:** Translate these requirements into user stories that describe the functionality from an end-user perspective.

- **Specify Scenarios:** For each user story, specify scenarios in a Given-When-Then format. Each scenario outlines a specific case of the user story, detailing the initial context (**Given**), the event or action (**When**), and the expected outcome (**Then**).

Example: Let's consider a simple test that searches for images of water on Google. Below would be a reasonable test procedure:

Step	Procedure	Response
1	Open a web browser	A web browser opens successfully
2	Navigate to https://www.google.com/	The web page loads successfully, and the Google image is visible
3	Enter "water" in the search bar	Links related to "water" are shown on the results page
4	Click on the "Images" link at the top of the results page	Images related to "water" are shown on the results page

To translate a test like this into Gherkin we must remember that **one scenario will only cover one behavior**. Therefore, we have two scenarios in the test following the Given-When-Then ordering, like this:

```
Feature: Google Searching

   Scenario: Search from the search bar
       Given a web browser is at the Google home page
       When the user enters "water" into the search bar
       Then links related to "water" are shown on the results page

   Scenario: Image search
       Given Google search results for "water" are shown
       When the user clicks on the "Images" link at the top of the results page
       Then images related to "water" are shown on the results page
```

Arguably, the first behavior scenario must run before the second, as the latter requires beginning at the search result page. However, the separation of these scenarios guarantees behavior-level independence, which ultimately represents software requirements (user stories) and acceptance criteria. We can directly write the Gherkin instructions in the code interface of **Conformiq Visualizer**:

Code Views

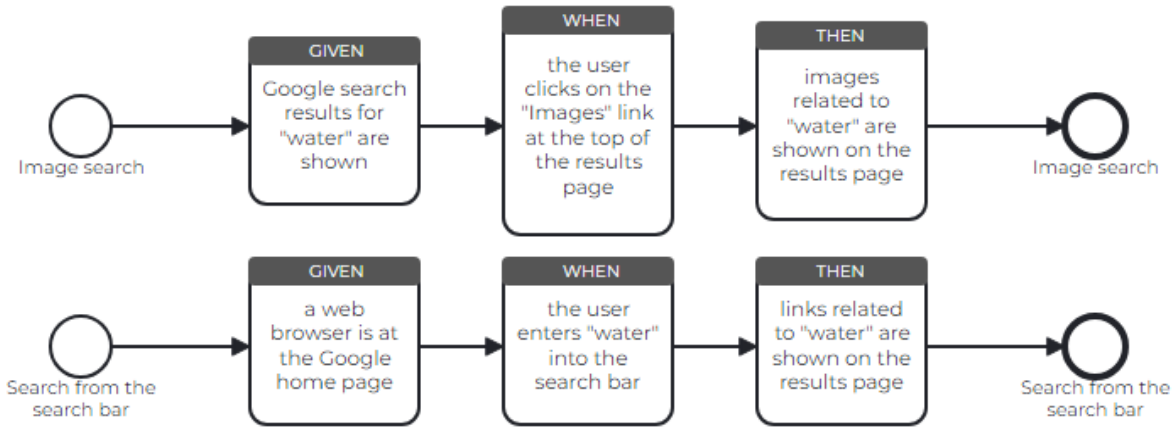
```
Feature: Google Searching

    Scenario: Search from the search bar
        Given a web browser is at the Google home page
        When the user enters "water" into the search bar
        Then links related to "water" are shown on the results page

    Scenario: Image search
        Given Google search results for "water" are shown
        When the user clicks on the "Images" link at the top of the results page
        Then images related to "water" are shown on the results page

```

> *The Golden Gherkin Rule: Write Gherkin in a way that treats other readers as you would want to be treated, ensuring that even those unfamiliar with the feature can understand it*

2. Create Models in Conformiq Visualizer

- **Model the System:** Use Conformiq Visualizer to create a visual model of the software system. This involves defining the components, interfaces, and interactions that represent the system's architecture and expected behaviors.
- **Incorporate Scenarios:** Integrate the BDD scenarios into the model. This might involve setting up specific states or transitions in the model that correspond to the Given, When, and Then steps.
- **Validate the Model:** Ensure the model accurately reflects the BDD scenarios and covers all the functional aspects described in the user stories.

Example: From the previous example of a test that searches for images of water on Google. Below is the model that Conformiq Visualizer creates automatically after adding the Gherkin code:



Conformiq Visualizer also allows you to design the model first and it will automatically write the Gherkin code for you. The models are created in the BPMN standard, which visually represents business processes that enable you to analyze your workflows.

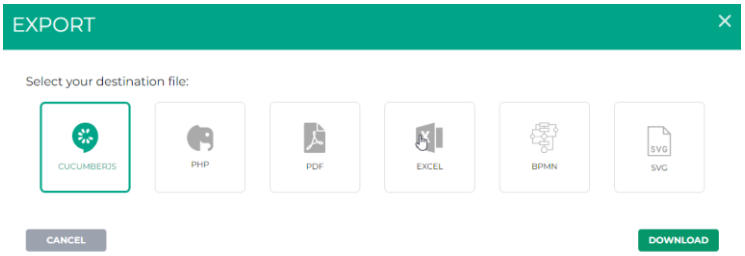
A standard Business Process Model and Notation (BPMN) allows businesses to map out every step of their operations by using graphical models that everyone can read and understand. It takes a process-oriented approach, helping team members to quickly and easily adapt to new changes (either internal or B2B).

> *BPMN models make it easier to see how different organizations, users, and service providers work together and manage transactions*

3. Export your Test Case Scenarios

- **Automate Test Generation:** Use Conformiq Visualizer to automatically generate test cases from the graphical model. This software can interpret the scenarios modeled and produce detailed test cases that include necessary inputs, execution steps, and expected outcomes.
- **Review Test Coverage:** Analyze the generated test cases to ensure they cover all the scenarios defined in the BDD process. Adjust the model as needed to fill any gaps.

Example: Let's take a look at the previous example again. **Conformiq Visualizer** allows us to import files with Gherkin code, to see it represented in the interface. However, when our goal is to generate test cases from the models. We can simply export the Gherkin code into different formats such as:



- ✓ Cucumber JS
- ✓ PHP
- ✓ PDF Files
- ✓ Excel
- ✓ BPMN Export
- ✓ SVG Export

If we select a PDF file, for instance, then we will get all the test cases to be covered. This tool is great when we are presenting large test cases in **Acceptance Test Plans** in front of a customer. Remember that we can also export the BPMN models, which would be great to introduce a feature to a non-technical audience, such as Project Managers or Final Users.

Test suite

Test Suite Summary

Test Case	Name	Summary	Verdict	Observation
1	Search from the search bar	Search from the search bar		
2	Image search	Image search		

Scenario 1: Search from the search bar
Summary: Search from the search bar

Step	Action	Verification point(s)	Verdict	Observation
1	Given a web browser is at the Google home page When the user enters "water" into the search bar	Then links related to "water" are shown on the results page		

Scenario 2: Image search
Summary: Image search

Step	Action	Verification point(s)	Verdict	Observation
1	Given Google search results for "water" are shown When the user clicks on the "Images" link at the top of the results page	Then images related to "water" are shown on the results page		

4. Insert Datasets into your Models

- **Accurate Test Cases**: This functionality allows users to import, create, and use specific data sets directly into their models for more accurate and comprehensive testing scenarios. Through **Drag & Drop** predefined data, you can ensure that the generated test cases are robust, covering various scenarios and edge cases that align with real-world data.
- **Data Centralization:** Central storage of all necessary data within the application, similar to a main desk in a library where all key information is kept. Gherkin files act like bookshelves containing scenarios (books) that describe different behaviors of the application, all based on the central data.

Example: Let's consider an example in which data would be necessary for testing, such as the case of a user login scenario. After translating it into Gherkin, we should be able to describe the steps when a user tries to log into a website, as follows:

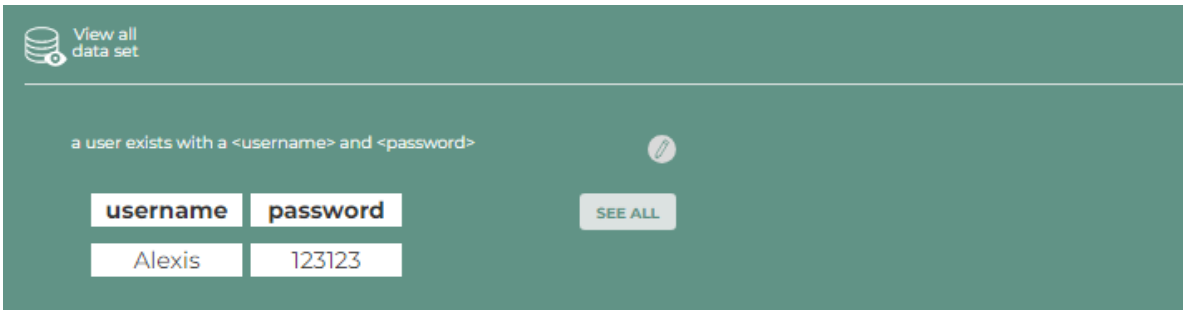
```
Feature: User login in a website

   Scenario: Valid user login
       Given a user exists with a <username> and <password>
         |username|password|
         |Alexis|123123|
       When the user tries to log in with the correct credentials
       Then the user should be successfully logged in
```

The *username* and *password* data would be linked to the centralized data storage, which guarantees consistency and simplifies maintenance. When you update a single piece of data only once, every associated Gherkin file automatically reflects this change. It makes your system's BDD more efficient and manageable. In the code interface of **Conformiq Visualizer**, our instructions would look like this:

Code Views

```
1 Feature: User login in a website
2
3     Scenario: Valid user login
4         Given a user exists with a <username> and <password>
5             | username | password |
6             | Alexis   | 123123   |
7         When the user tries to log in with the correct credentials
8         Then the user should be successfully logged in
9
```

The data introduced in the model can be reviewed, updated, and exported:



5. Create Test Automation Files

- **Automate Test Execution**: The test cases generated by Conformiq Visualizer can be exported into automation script files. You can integrate these tests with your existing test execution environment or test automation frameworks.
- **Collect and Analyze Results**: Gather results from test executions to verify that the system behaves as expected across all scenarios. BDD is an iterative process, so you must use any failures or defects as feedback to refine the model and test cases. Repeat the process to continuously improve the quality and relevance of the test cases.

Example: For any given test case scenario that you created, **Conformiq Visualizer** will give you the option to convert it into automation script files. In the Automation panel, the previous example will look as follows:

JS Support > **Login Attempt**

```
const { Given, When, Then }= require('@cucumber/cucumber');

// Scenario: Valid user login
Given('a user exists with a <username> and <password>', function (dataTable) {
//a user exists with a <username> and <password>
// Your test goes here ;
});

When('the user tries to log in with the correct credentials', function () {
// Your test goes here ;
});

Then('the user should be successfully logged in', function () {
// Your test goes here ;
});
```

6. Collaborate with your Team

- **Collaborate Across Teams**: Ensure that business analysts, developers, and testers are collaborating throughout the process, using the visual models and generated test cases as common reference points.

- **Integrate with Development Workflows**: Link the outcomes of your testing back into your development process, using tools like Jira or other ALM tools to track bugs, feature requests, and enhancements.

> *Using Conformiq Visualizer helps bridge the gap between traditional manual test case design and automated behavior-driven development processes, enhancing both the efficiency and effectiveness of testing in software development projects*

Overcoming BDD Deployment Challenges

Deploying BDD effectively across a project can be challenging due to issues like tool compatibility, test complexity, and team coordination. Conformiq Visualizer helps mitigate many of these challenges:

- One significant hurdle in BDD implementation is ensuring that the tools used are compatible with existing development and testing ecosystems. Conformiq Visualizer is designed to integrate smoothly with popular Application Lifecycle Management (ALM) tools such as Jira, GitHub, GitLab, Bitbucket, Azure DevOps, and Cucumber JS. It provides a seamless workflow from requirements management to test execution, which is critical for maintaining the BDD lifecycle.

- Another challenge is the automation of test execution and effective issue tracking. Conformiq Visualizer excels here by not only generating test cases automatically from the defined behavior models but also by facilitating the generation of detailed reports. These features enhance traceability and allow for quicker responses to issues that arise during testing phases, ensuring that defects are tracked and addressed efficiently, in line with BDD practices.

Conformiq Visualizer plays a pivotal role in realizing the full potential of BDD by bridging the gap between technical and non-technical team members, automating crucial testing processes, and ensuring high compatibility and integration with existing tools. Adopting Conformiq Visualizer not only simplifies the complexities associated with BDD but also elevates the overall efficiency and effectiveness of software development cycles.